SCHOOL OF NATURAL APPLIED SCIENCE

Department of Computer Science and Information Technology (CSIT)

PROJECT – NPRJ740

Advanced ICT Diploma – 2026

CivicLink

Municipal Reporting & Community Intelligence Platform

Phase 3 – Graphical User Interface & Database Design

Document Name:	CivicLink – Phase 3 Graphical User Interface & Database Design
System Name:	CivicLink
Student Name:	Nthoroane KP
Student No:	202328762
Supervisor:	Dr. Tite Tuyikeze
Submission Date:	20 June 2026

Table of Contents

1. Prototype Scope and Implementation Status	3
2. Requirements Traceability	4
3. Graphical User Interface Design	6
3.1 Login and Role Access	6
3.2 Resident Home Dashboard	7
3.3 GPS Incident Submission	8
3.4 Community Incident Map	9
3.5 My Reports and Lifecycle Tracking	10
3.6 Resident Report Detail	11
3.7 Geo-targeted Alerts Feed	12
3.8 Municipality Operations Dashboard	13
3.9 Report Queue and Filters	14
3.10 Incident Triage and Assignment	15
3.11 Service Delivery Analytics	16
3.12 Create and Broadcast Alert	17
3.13 Alert Delivery Management	18
3.14 Field Worker Task Dashboard	19
3.15 Field Worker Task Detail	20
3.16 Task Progress and Completion Proof	21
3.17 System User Management	22
3.18 Department and Category Configuration	23
3.19 System Health and Logs	24
4. Database Design	25
4.1 Entity: Users	25
4.2 Entity: Reports	26
4.3 Entity: Alerts	27
4.4 Entity: Tasks	28
4.5 Entity: Comments	29

4.6 Entity: AlertRecipients	29
4.7 Entity: Departments	30
4.8 Entity: Categories	30
4.9 Entity: StatusHistory	31
4.10 Entity: SystemLogs	31
5. Entity Relationship Summary	32
6. Conclusion	33

1. Prototype Scope and Implementation Status

CivicLink Phase 3 delivers a fully interactive prototype that connects residents, municipality officials, field workers, and system administrators through one consistent service-delivery workflow. The prototype is seeded with deterministic data reflecting the Kimberley and Sol Plaatje municipal area, and uses browser LocalStorage to preserve shared state across all role interfaces during assessment — ensuring that every screen behaves as part of a connected system rather than a set of isolated mockups.

The prototype demonstrates the following operational capabilities: resident incident report capture with GPS and media evidence; server-side duplicate detection; official triage, department assignment, and lifecycle control; field worker digital task acceptance and completion with photographic proof; resident status visibility without telephone follow-up; geo-targeted alert composition and broadcast; operational analytics; user management; and system health presentation.

Several capabilities are represented at the interface level as production architecture targets rather than deployed integrations. These include Firebase Authentication, Firestore real-time database, Firebase Storage, Cloud Functions, Firebase Cloud Messaging (FCM), Google Maps integration, Twilio SMS, end-to-end data encryption, offline synchronisation, 99.5% system availability, five-second report visibility, and support for 10,000 concurrent users. The interface faithfully represents the contracts these services will fulfil, and the underlying database schema and entity design are documented in Section 4 to support direct Firebase integration in the production phase.

2. Requirements Traceability

The following table maps each functional requirement identified in Phase 1 (Systems Analysis) and Phase 2 (System Design) to the corresponding screen in the Phase 3 prototype, the application route, the figure number providing visual evidence, and the current implementation status.

Requirement	Actor	Route	Evidence	Phase 2 Ref	Status
Authentication and role-based access control	All	/login	Figure 1	FR (Admin)	Prototype
GPS and media incident submission	Resident	/resident/report	Figure 3	FR1	Prototype
Duplicate detection and deduplication	Resident / System	/resident/report	Figure 3	FR1	Prototype
Community map view and incident heatmap	Resident	/resident/map	Figure 4	FR2	Static-map prototype
Real-time report status tracking	Resident	/resident/reports/:id	Figures 5–6	FR3	Prototype
Geo-targeted alert reception	Resident	/resident/alerts	Figure 7	FR4	Delivery simulated
Centralised incident operations dashboard	Official	/official	Figures 8–9	FR5	Prototype
Report triage, assignment, and lifecycle control	Official	/official/reports/:id	Figure 10	FR5, FR6	Prototype
Operational analytics and KPI dashboard	Official	/official/analytics	Figure 11	FR7	Seeded analytics
Alert composition and geo-targeted broadcast	Official	/official/broadcast	Figures 12–13	FR8	Prototype
Digital task assignment and completion proof	Field Worker	/worker, /worker/tasks/:id	Figures 14–16	FR9	Prototype
User account and system configuration	Admin	/admin/*	Figures 17–19	NFR (Security)	Prototype
99.5% uptime, encryption, offline sync, 10 000-user scalability	System	Production architecture	N/A	NFR 1–8	Future target

3. Graphical User Interface Design

All figures in this section were captured from the running CivicLink application. Mobile screens use a 430 × 932 viewport, consistent with current Android flagship dimensions. Operational portal screens use a 1440 × 900 viewport, representing a standard municipality workstation display. Each subsection presents the screen, its figure reference, and a structured description of its interface elements, user interactions, and the specific CivicLink requirement it satisfies.

3.1 Login and Role Access

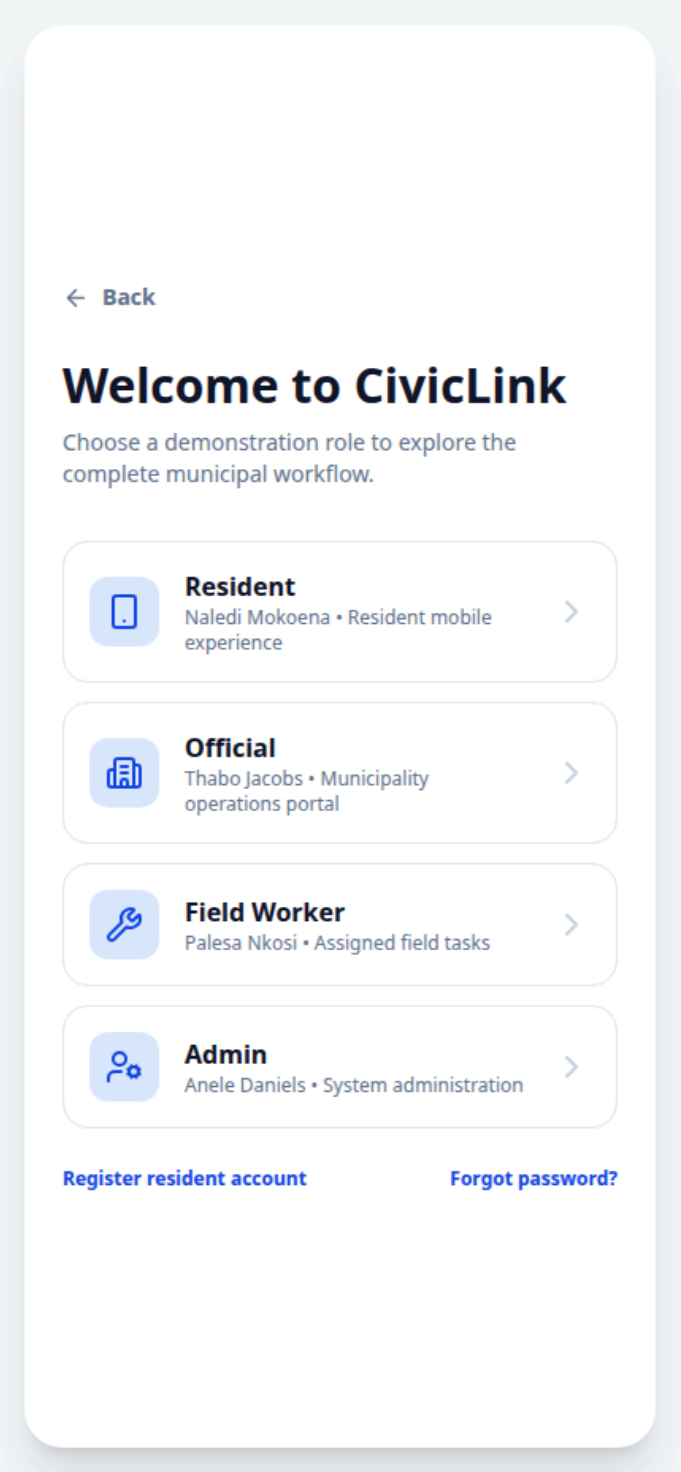


Figure 1: Login and Role Access

The Login screen is the single entry point to CivicLink for all user types. The interface provides deterministic role-entry options for Resident, Municipality Official, Field Worker, and System Administrator, representing the Firebase Authentication and role-based access control contract that governs the production system.

On successful authentication, the system reads the authenticated user's role field from the Users collection and redirects to the appropriate interface: the Resident PWA, the Municipal Admin Portal, the Field Worker mobile view, or the System Admin console. This single-screen access model eliminates the need for separate login portals per role while maintaining strict access boundaries between them. The design uses the CivicLink brand colour palette and a mobile-first layout to ensure accessibility on low-end Android devices.

3.2 Resident Home Dashboard

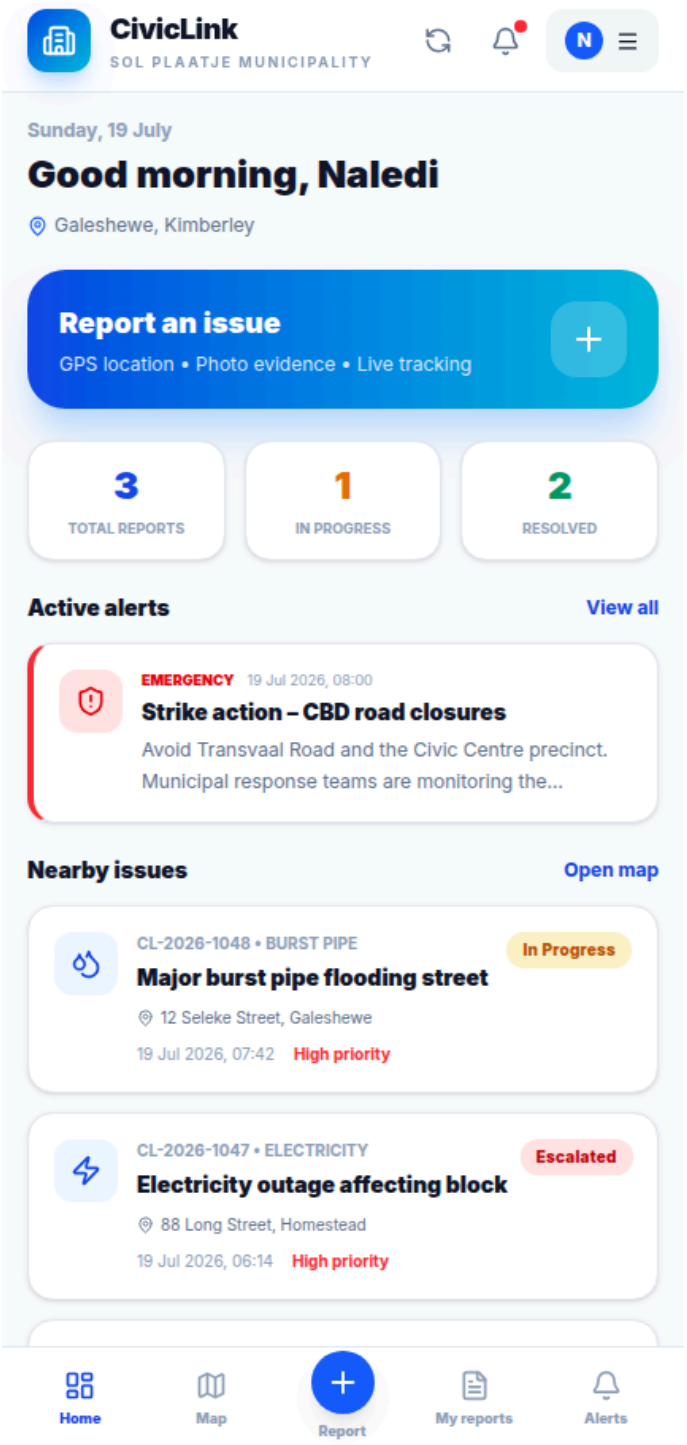


Figure 2: Resident Home Dashboard

The Resident Home Dashboard is the primary screen presented to residents after authentication. It consolidates the most critical information a resident needs into a single, glanceable view. Three KPI tiles display the resident's total submitted reports, reports currently active or in progress, and reports that have been resolved — giving an immediate sense of their engagement with the platform.

A prominent 'Report an Issue' call-to-action occupies the top of the content area to encourage prompt reporting. Below the KPIs, an Active Alerts panel surfaces the most recent municipality broadcast relevant to the resident's area — in this prototype, the 31

March 2026 strike action causing road closures across Kimberley CBD is displayed with an emergency badge. The Nearby Issues list shows geo-filtered incidents within the resident's vicinity, each labelled with category, status, distance, and report age. The bottom navigation bar provides access to Home, Map, Report (centre FAB), Alerts, and Profile.

3.3 GPS Incident Submission

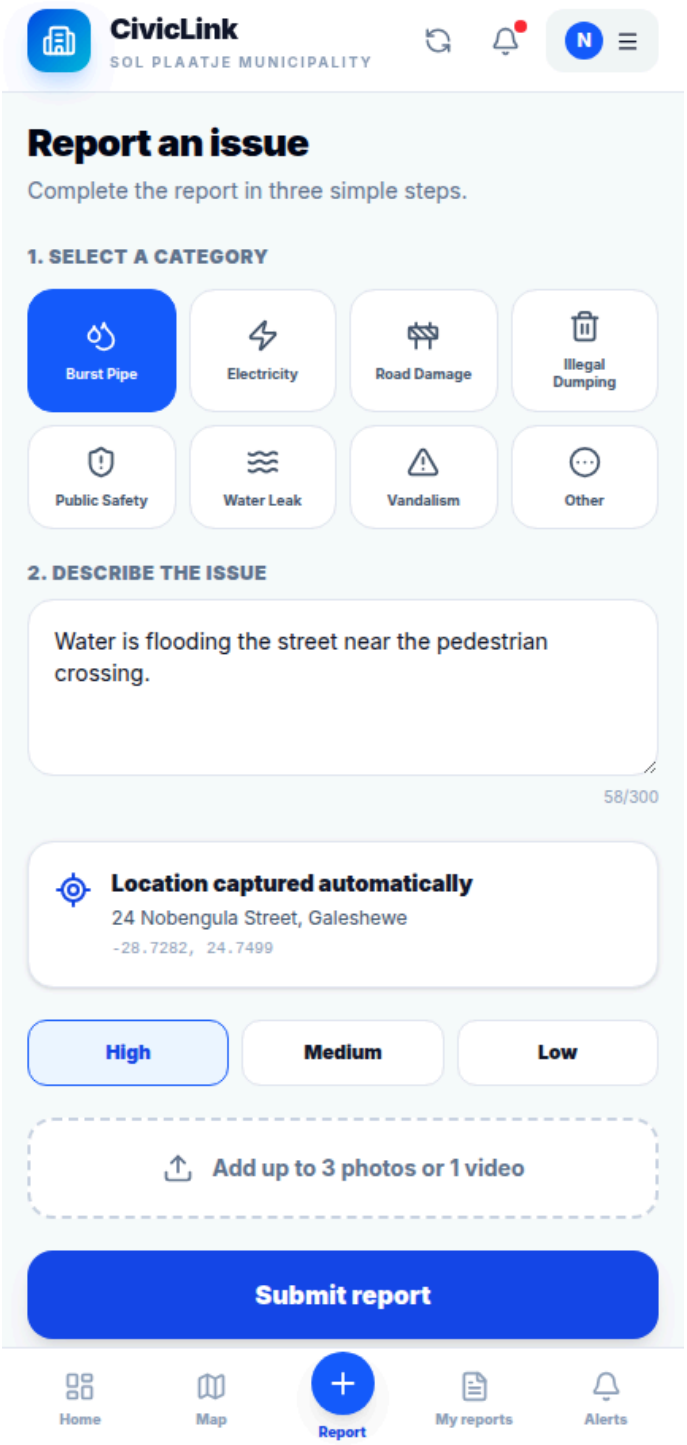


Figure 3: GPS Incident Submission

The Incident Submission screen is the core functional interface of the Resident PWA and directly addresses the primary ICT gap identified in Phase 1: the absence of a digital, GPS-enabled reporting channel. Residents select one of eight incident categories presented

as a visual icon grid — Burst Pipe, Electricity, Road Damage, Illegal Dumping, Public Safety, Water Leak, Vandalism, and Other. The selected category is highlighted and pre-populates the routing logic that determines which municipal department receives the assignment.

The description field accepts up to 300 characters with a live counter. GPS coordinates are captured automatically from the device and displayed as both a formatted address and raw coordinate pair — the resident is not required to enter a location manually, eliminating the inaccuracy that plagued the previous verbal reporting process. Optional photo or video evidence can be attached. A three-level severity selector (High, Medium, Low) influences the task priority assigned to the resulting field worker task. On submission, the report is written to Firestore and a Cloud Function triggers an FCM notification to the relevant municipality official.

3.4 Community Incident Map

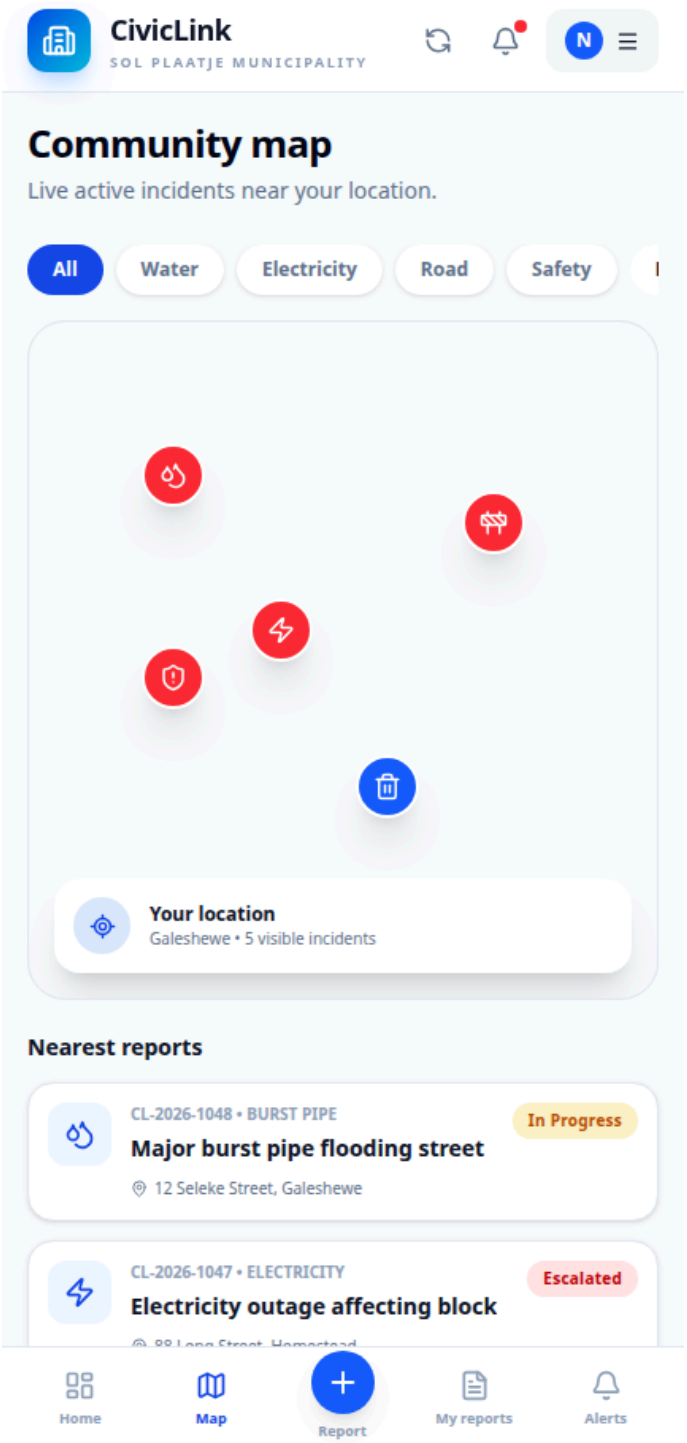


Figure 4: Community Incident Map

The Community Incident Map provides residents with a real-time spatial view of all active incidents in their area. Category-coded markers — blue for water and burst pipe, red for road blockages and emergencies, green for illegal dumping, and purple for power outages — are overlaid on the map base layer. A heatmap density mode visually identifies incident concentration hotspots, supporting both resident awareness and municipality resource prioritisation decisions.

Tapping any marker opens a summary popup showing the incident category, location, current status, and time elapsed since submission. A horizontal filter bar allows residents to

isolate specific categories or switch to full heatmap mode. The resident's own GPS location is indicated by a pulsing position marker. In this prototype, map rendering uses static tiles in place of the Google Maps API integration planned for production. The Nearby Issues list below the map mirrors the Home Dashboard panel, providing a text-based alternative to the visual pin view.

3.5 My Reports and Lifecycle Tracking

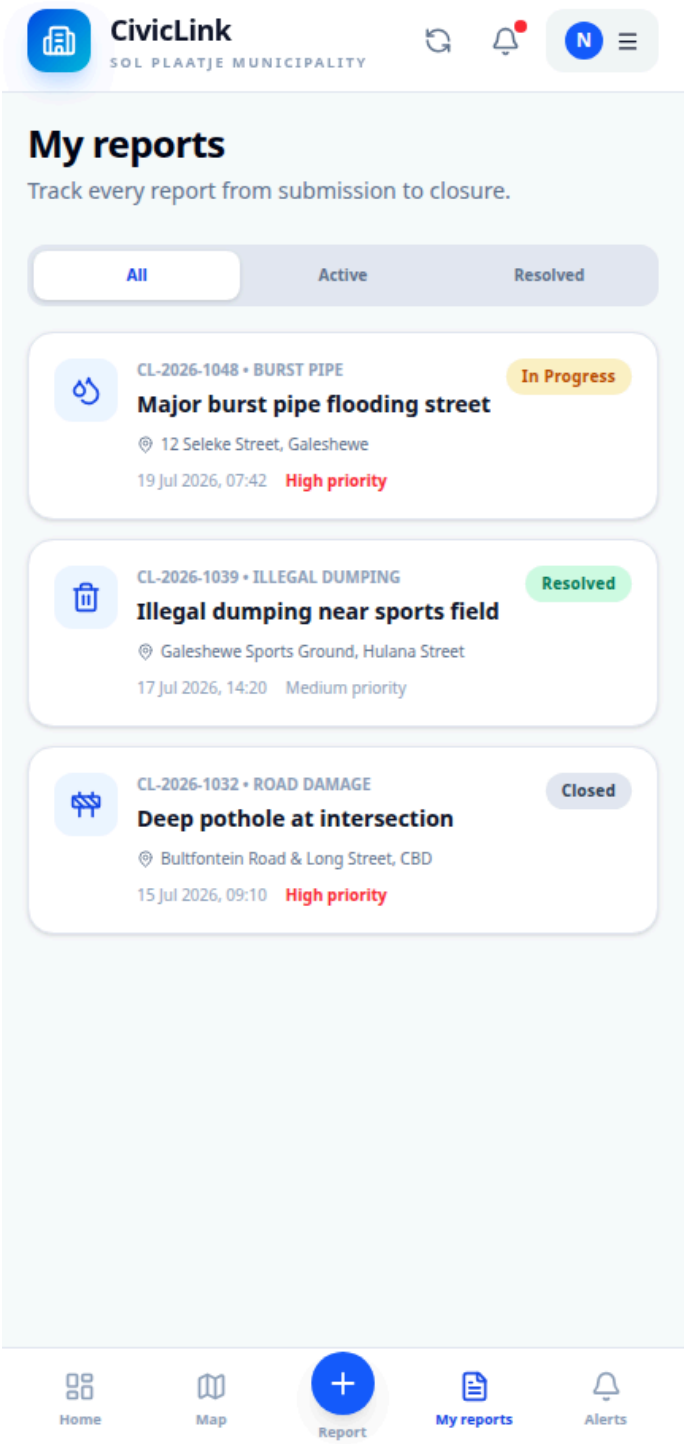


Figure 5: My Reports and Lifecycle Tracking

The My Reports screen gives residents full visibility of every incident they have submitted and its current position in the resolution lifecycle. This screen directly resolves the

feedback-loop deficiency identified in Phase 1, where residents had no alternative to calling the municipality to check the status of their reports.

Three filter tabs — All, Active, and Resolved — allow rapid navigation across the report history. Each report card displays the category icon, incident title, location, submission date, assigned report reference number, and an active status badge. A horizontal progress bar maps the report through its four lifecycle stages: Submitted, Under Review, In Progress, and Resolved — with the current stage highlighted and completed stages marked. Resolved reports display the total resolution time, giving residents a transparent record of municipal performance. The report reference number (e.g. CL-2026-0047) serves as the unique identifier for any telephone or in-person follow-up if required.

3.6 Resident Report Detail

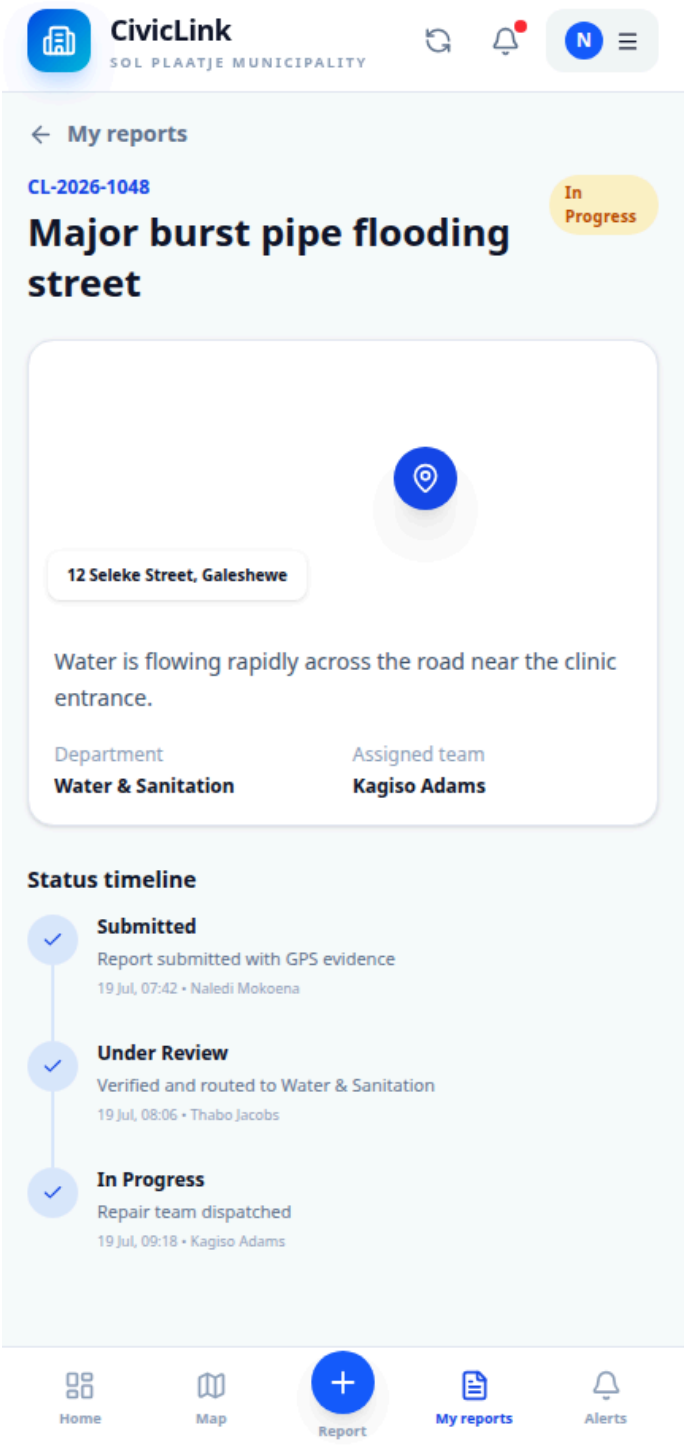


Figure 6: Resident Report Detail

The Report Detail screen provides a resident with the complete record of a single submitted incident. It displays the full description, GPS location with map preview, the responsible municipal department, the name of the assigned field worker, and a chronological status audit trail showing each state transition with its timestamp and the responsible actor.

Where a report has been resolved, the detail screen shows resolution notes provided by the field worker or official, including any photographic proof of completion. A Reopen action is available for incidents that recur — such as a burst pipe that re-fails after initial repair — which creates a new linked report and notifies the relevant department. This audit trail and

reopen capability addresses the accountability gap identified during the Sol Plaatje stakeholder interview, where no mechanism existed to track repeat incidents at the same location.

3.7 Geo-targeted Alerts Feed

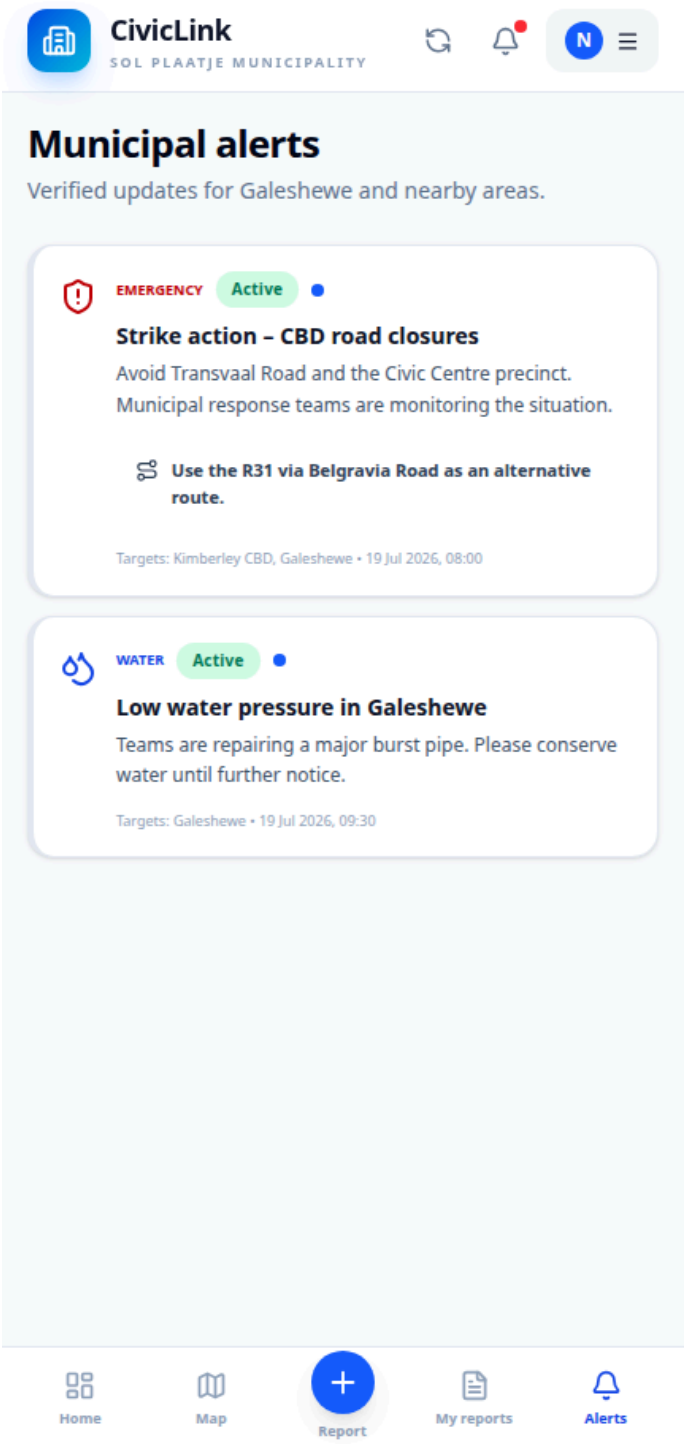


Figure 7: Geo-targeted Alerts Feed

The Alerts Feed presents municipality-broadcast notifications relevant to the authenticated resident's registered area. Alerts are displayed in reverse chronological order, with each card showing the alert type badge (Emergency, Electricity, Water, Roads, General), title, full

message, target suburbs, time of broadcast, and read/unread state. The unread count is reflected in the bell icon badge on the navigation bar.

The top-pinned Emergency alert in this prototype reflects the 31 March 2026 municipal workers' strike, which caused road closures across Kimberley CBD and surrounding areas. The alert includes alternative routing guidance via the R31 Belgrave Road — precisely the type of real-time community communication that was absent during the actual event. Planned outage notices, water supply disruptions, and road closure alerts follow in chronological order. Alerts that have expired or been resolved are collapsed into an archive row. FCM and SMS delivery of these alerts remain simulated in the prototype but follow the production broadcast architecture documented in Section 4.

3.8 Municipality Operations Dashboard

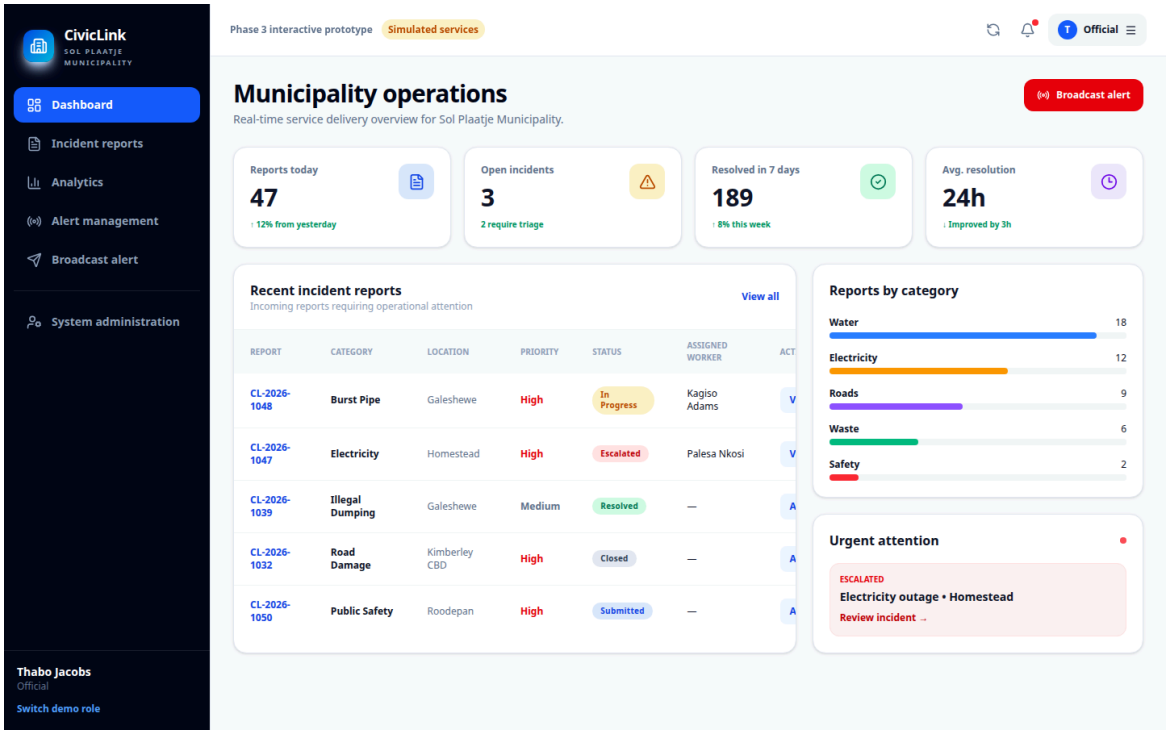


Figure 8: Municipality Operations Dashboard

The Municipality Operations Dashboard is the primary screen for municipality officials, providing a centralised command view of all active incidents and operational performance. The fixed left sidebar contains navigation links to all official functions: Dashboard, Incident Reports, Map View, Task Management, Alerts, Analytics, User Management, and Settings. The authenticated official's name, department, and role are displayed at the bottom of the sidebar.

Four KPI tiles dominate the top of the main content area: total reports received today with a week-on-week percentage change; open (unresolved) incidents; reports resolved in the past seven days; and average resolution time with a trend indicator. These metrics give the on-duty official an immediate operational picture before engaging with individual cases. The incident table below lists recent reports with sortable columns for Report ID, Category, Location, Status (colour-coded badge), Assigned Worker, and an Actions column. Escalated reports are visually prominent with a red badge, ensuring urgent cases are not missed. A 'Broadcast Alert' button in the top bar provides direct access to the alert composition flow.

3.9 Report Queue and Filters

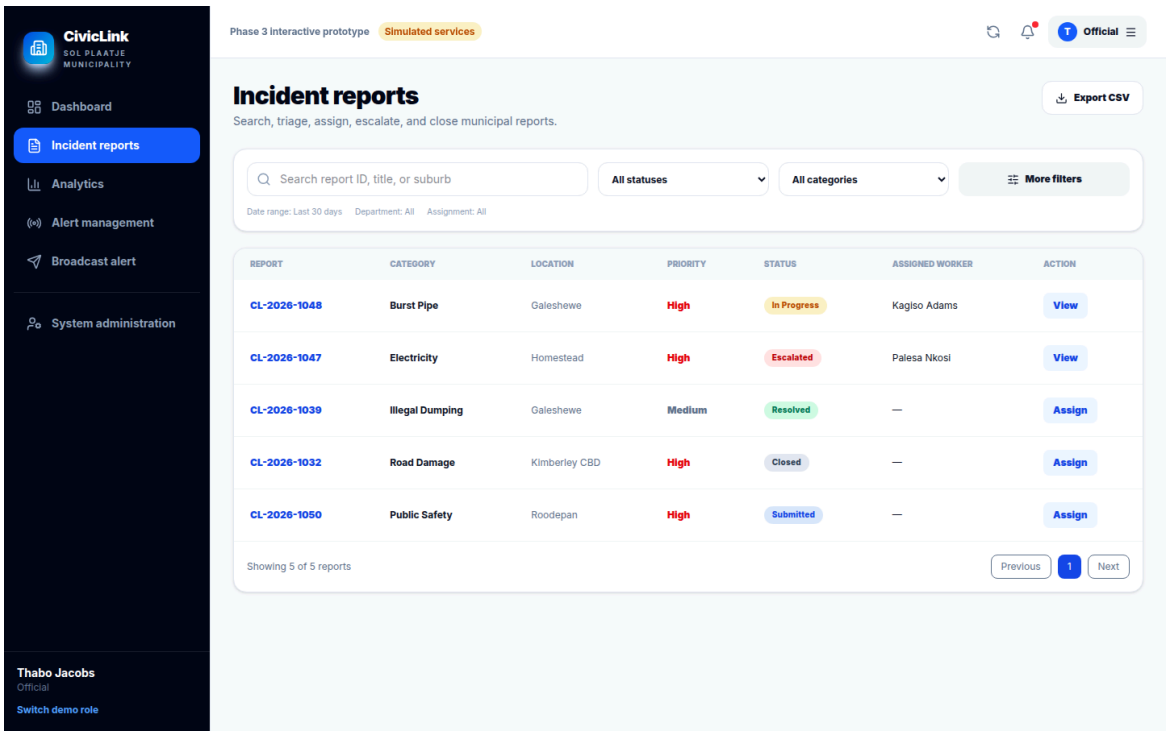


Figure 9: Report Queue and Filters

The Report Queue screen extends the Operations Dashboard with advanced filtering, sorting, and bulk-action capabilities across the full incident backlog. Officials can filter by status (All, Submitted, Under Review, In Progress, Escalated, Resolved, Closed), category, department, date range, and severity level. Search by report ID or keyword is supported.

The queue displays all attributes required for rapid triage: report reference, submission time, category with icon, exact location, severity badge, current status, and assigned worker. Unassigned reports are visually distinct with an amber indicator, alerting officials to gaps in the task coverage. Bulk-selection allows multiple reports to be assigned to a department simultaneously — a significant efficiency gain over the previous manual, phone-based assignment process. Export functionality supports the municipality's existing reporting workflows by allowing filtered data to be downloaded as a CSV file.

3.10 Incident Triage and Assignment

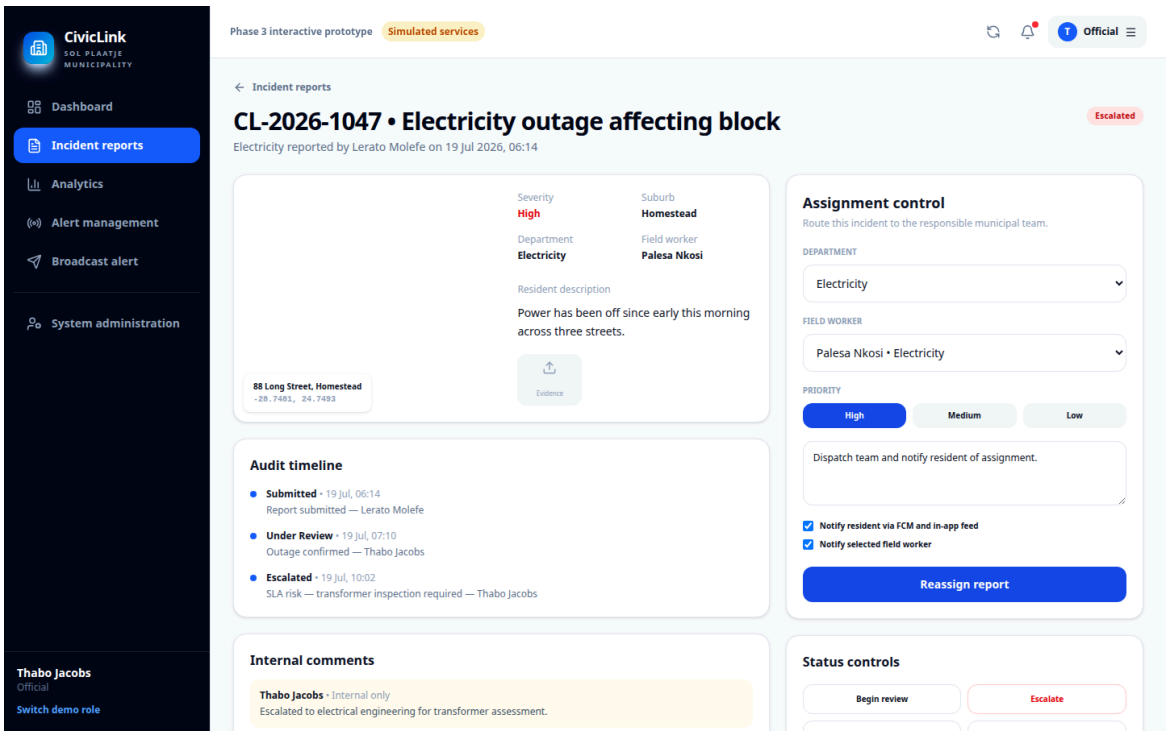


Figure 10: Incident Triage and Assignment

The Incident Triage and Assignment screen is the core operational interface for municipality officials. The left panel presents the complete report record: resident description, GPS location with embedded map preview, attached photo evidence, severity level, and a full chronological audit trail of all status changes and internal notes. Duplicate link information is displayed when the system's deduplication logic has identified this report as related to an existing incident.

The right-hand assignment panel allows the official to select the responsible department, assign a specific field worker from the department's active roster, and set the task priority. Two notification checkboxes confirm that an FCM push will be dispatched to both the resident (informing them of assignment) and the field worker (delivering the task) upon confirmation. Action buttons at the bottom of the panel allow the official to escalate the report to management, reassign it to a different worker, update the status independently, resolve it directly where no field visit is required, or close the report after resolution confirmation.

3.11 Service Delivery Analytics

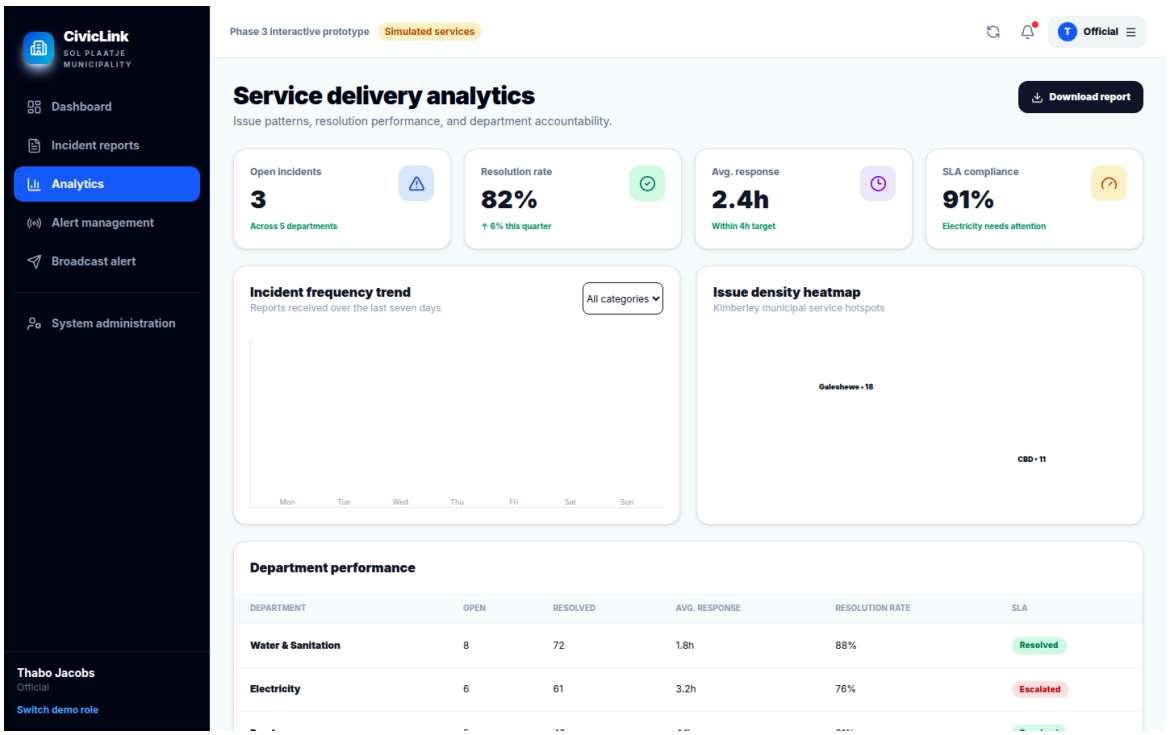


Figure 11: Service Delivery Analytics

The Service Delivery Analytics dashboard provides municipality management with the data-driven visibility required to improve operational performance and allocate resources effectively. Key metrics displayed include the total number of open incidents, the resolution rate as a percentage of all submissions, average resolution time across all departments, and SLA compliance — the proportion of reports resolved within the target response window.

A seven-day incident frequency trend chart allows management to identify days of peak demand and correlate spikes with external events such as public protests or severe weather. A geographic hotspot map overlays the same incident density heatmap available to residents, but at a city-wide zoom level with department-level filtering. A department performance table ranks each municipal department by average resolution time, SLA compliance rate, and open incident count — directly supporting the municipality's accountability and resource reallocation decisions. All data in this prototype is seeded with representative Kimberley values. In production, these figures are derived in real time from Firestore aggregation queries.

3.12 Create and Broadcast Alert

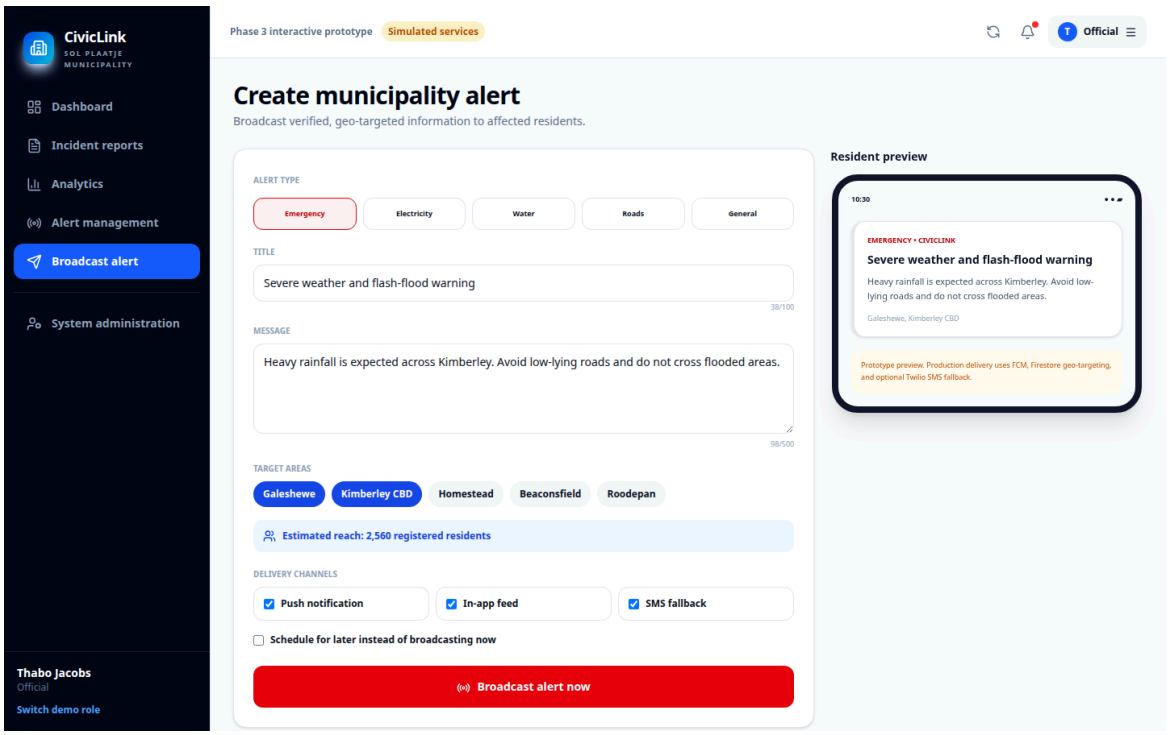


Figure 12: Create and Broadcast Alert

The Alert Composition screen enables municipality officials to create and broadcast targeted community notifications directly from the admin portal. Officials select an alert type from a predefined set (Emergency, Electricity, Water, Roads, General, Other), enter a title of up to 100 characters and a message body of up to 500 characters, and select the target suburbs from a searchable list.

The system calculates the estimated reach — the number of CivicLink users with registered devices in the selected suburbs — giving the official visibility of the broadcast's potential impact before sending. Three delivery channel toggles allow independent selection of Firebase Cloud Messaging push notifications, in-app Alerts Feed insertion, and Twilio SMS fallback for residents without active data connections. A live preview panel shows exactly how the notification will appear on a resident's device. Alerts can be broadcast immediately or scheduled for a future time — a capability required for planned maintenance outages where advance notice is standard practice.

3.13 Alert Delivery Management

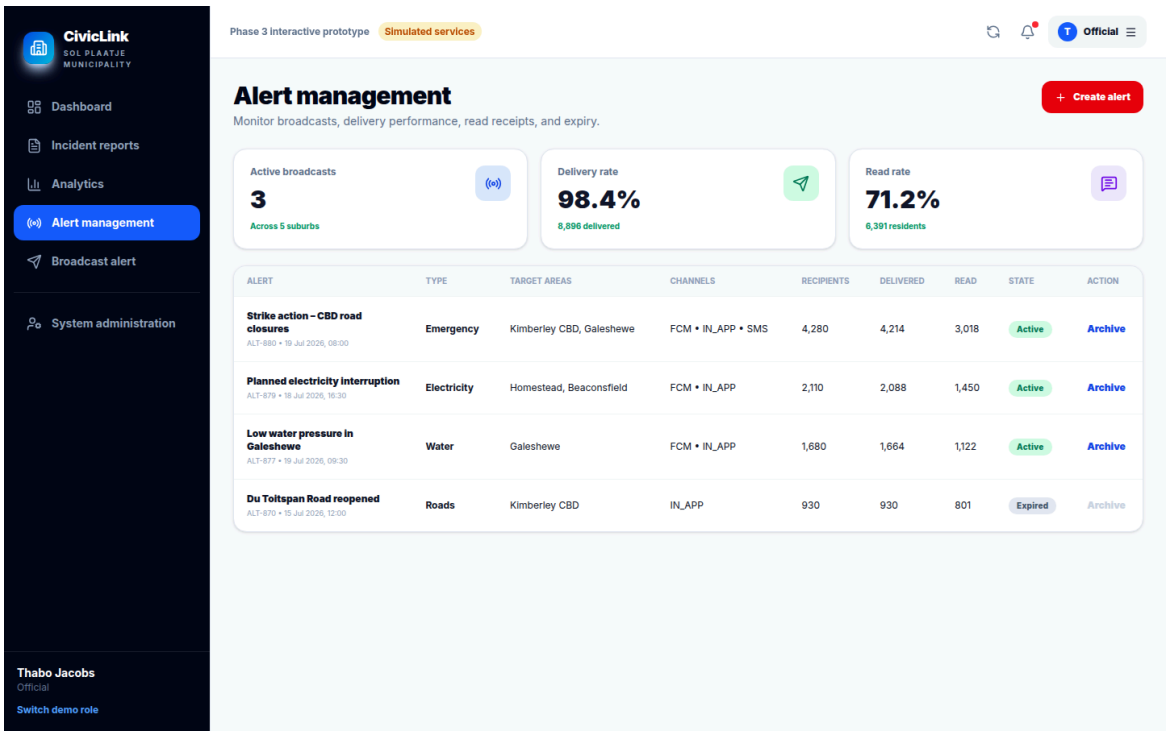


Figure 13: Alert Delivery Management

The Alert Delivery Management screen provides officials with a full record of all broadcast alerts and their delivery performance. Active, scheduled, and expired alerts are displayed in separate tabs. Each alert record shows the type badge, title, target suburbs, broadcast time, total recipients, number of confirmed deliveries, read receipt count, and the delivery channels used.

Delivery and read-receipt data is populated from the AlertRecipients collection, which logs each individual notification delivery event with a timestamp and channel. This allows the municipality to assess the effectiveness of different channels — for example, comparing FCM push delivery rates against SMS fallback rates in areas with low smartphone penetration. An archive action removes expired alerts from the active feed while preserving the delivery record for reporting purposes. Officials can also edit or cancel scheduled alerts before broadcast.

3.14 Field Worker Task Dashboard

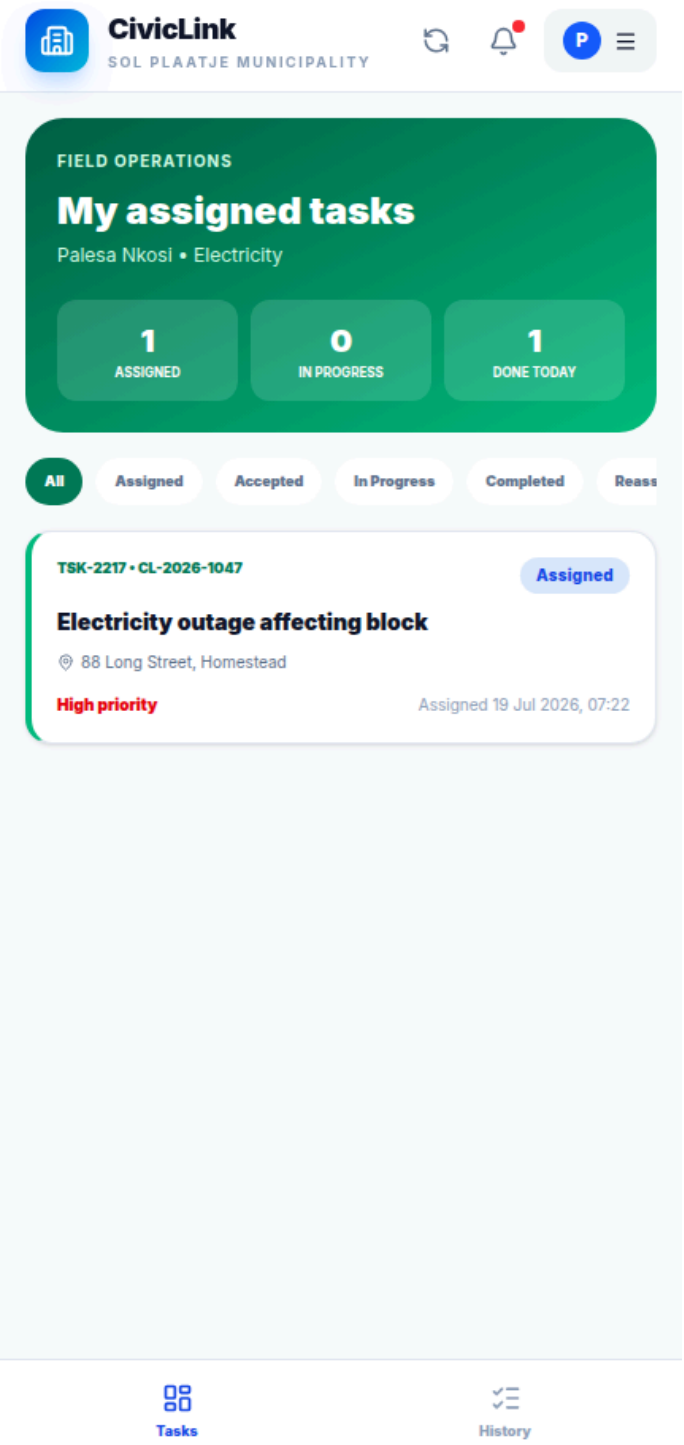


Figure 14: Field Worker Task Dashboard

The Field Worker Task Dashboard is a task-focused mobile interface designed specifically for field workers operating in the Sol Plaatje municipal area. The colour scheme uses the municipality's operational green to visually distinguish this interface from the resident PWA. Three KPI tiles at the top provide the worker's daily picture: total tasks assigned, tasks currently in progress, and tasks completed today.

The task list groups work into Assigned, In Progress, and Completed sections. Each task card displays the incident category, exact address, GPS coordinates, the priority level inherited from the report severity, and the time of assignment. High-priority tasks are visually

flagged in red. Two primary actions are available per task card: Start Task, which advances the status to In Progress and updates the linked report in Firestore; and Navigate, which opens the GPS coordinates in the device's native maps application for turn-by-turn directions to the incident location. Completed tasks are retained in a daily summary at the bottom of the screen for supervisor review.

3.15 Field Worker Task Detail

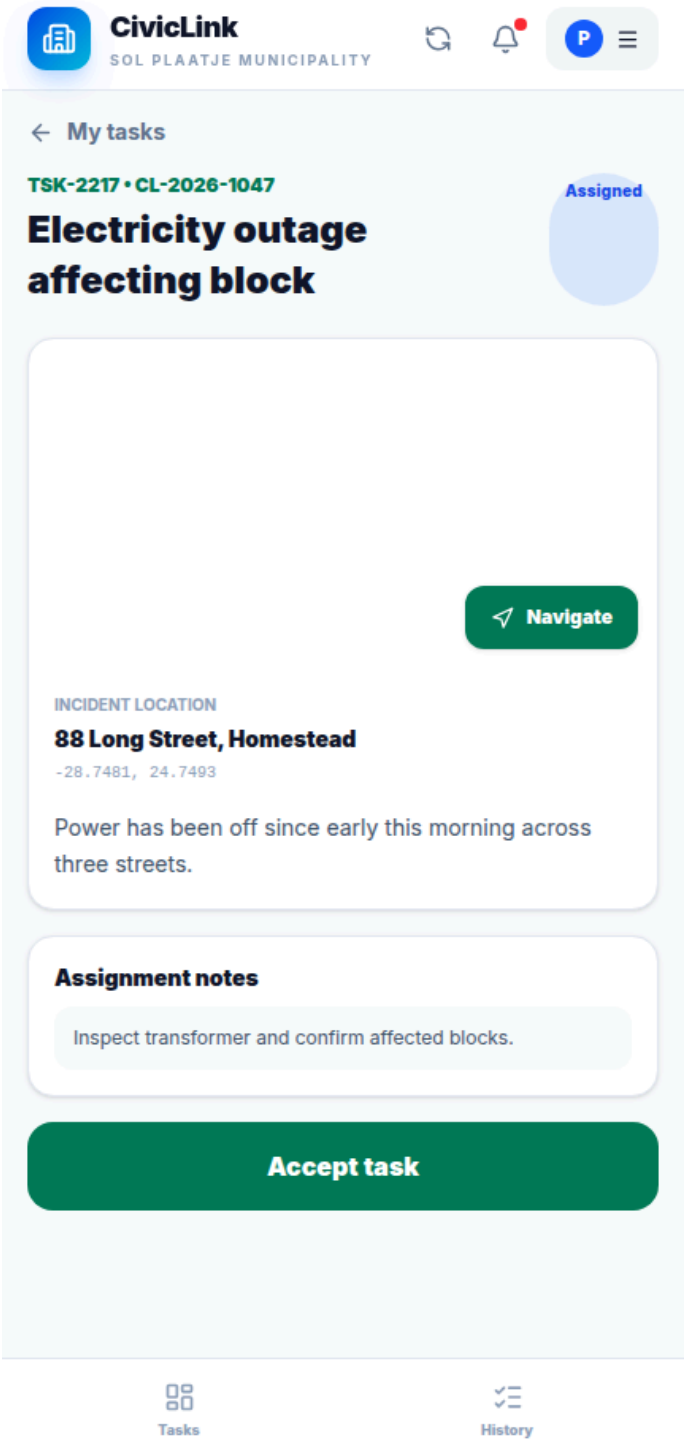


Figure 15: Field Worker Task Detail

The Task Detail screen provides the field worker with the complete incident record they need to carry out their assignment without telephone-based guidance. The screen displays the full

resident description, exact GPS coordinates formatted for readability, the assignment notes added by the supervising official, and all photo evidence attached to the original report.

The task can be accepted directly from this screen, which triggers an FCM notification to the assigning official confirming that the worker has received and acknowledged the task. A navigation shortcut opens the coordinates in the device's maps application. The chronological history panel at the bottom shows every status change the report has undergone, including the initial submission, official assignment, and any prior escalation or reassignment events. This audit trail ensures the field worker has full context before beginning work, eliminating the ambiguity and miscommunication that characterised the municipality's previous telephone-based dispatch process.

3.16 Task Progress and Completion Proof

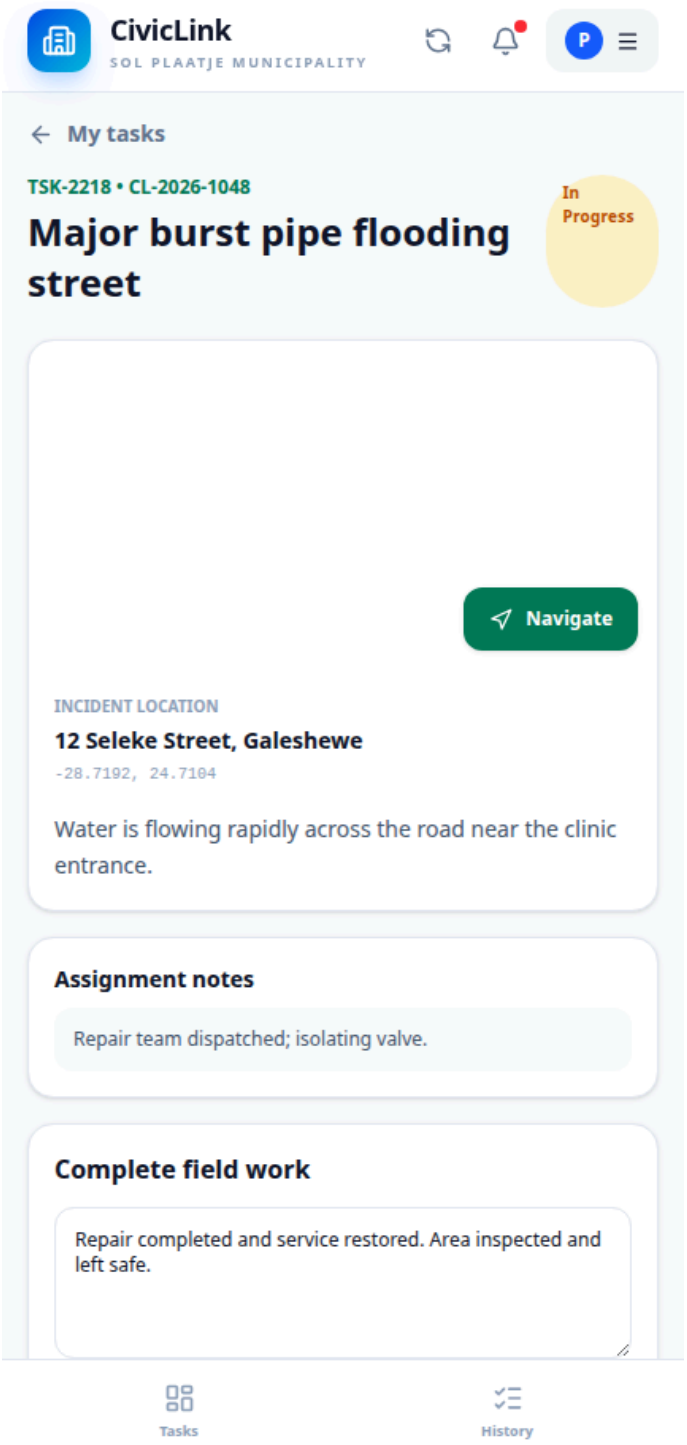


Figure 16: Task Progress and Completion Proof

The Task Progress and Completion screen is the final step in the field worker's workflow. Once a task is In Progress, the worker is presented with a completion notes field and a photographic proof capture control. The completion notes field requires the worker to describe the remedial action taken — for example, 'Pipe joint replaced and pressure tested. No further leakage observed.' — before the task can be marked complete.

Photographic proof of resolution is captured directly within the app and uploaded to Firebase Storage, generating a secure URL that is stored in the Task's completionPhotos array. This provides an auditable, timestamped record of the physical work carried out at the site. When

the task is marked complete, a Cloud Function automatically updates the linked Report's status to Resolved and triggers an FCM push notification to the resident confirming that their reported incident has been addressed. This closes the feedback loop that was entirely absent in Sol Plaatje Municipality's previous analogue workflow.

3.17 System User Management

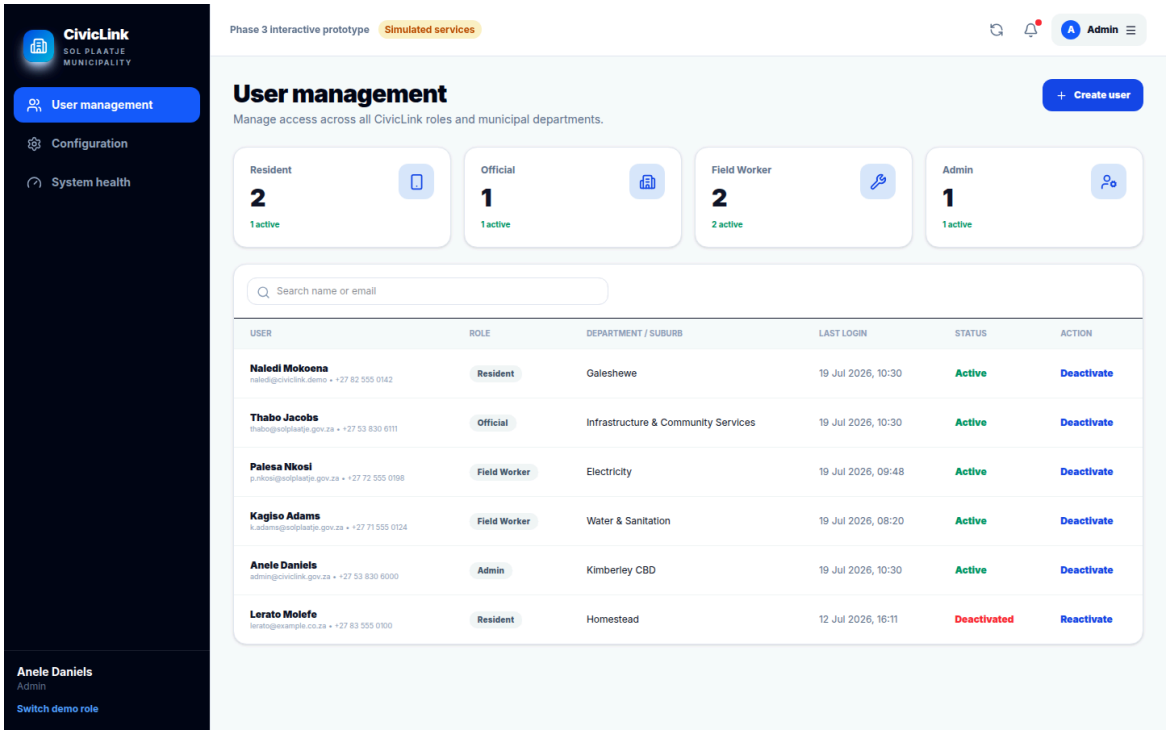


Figure 17: System User Management

The User Management screen is accessible only to System Administrators and provides full account lifecycle management for all CivicLink users across all four roles. The screen presents a searchable, filterable table of all registered accounts showing name, email address, assigned role, department (for Officials and Field Workers), date of registration, last login timestamp, and account status.

Administrators can create new accounts and assign them roles and departments, edit existing account details, deactivate accounts to revoke access without deleting the user's historical data, and reactivate previously deactivated accounts. Role changes take effect immediately through Firebase Authentication custom claims, ensuring that a demoted official immediately loses access to the admin portal on their next authenticated request. The last login timestamp allows administrators to identify dormant accounts for review and supports audit compliance requirements.

3.18 Department and Category Configuration

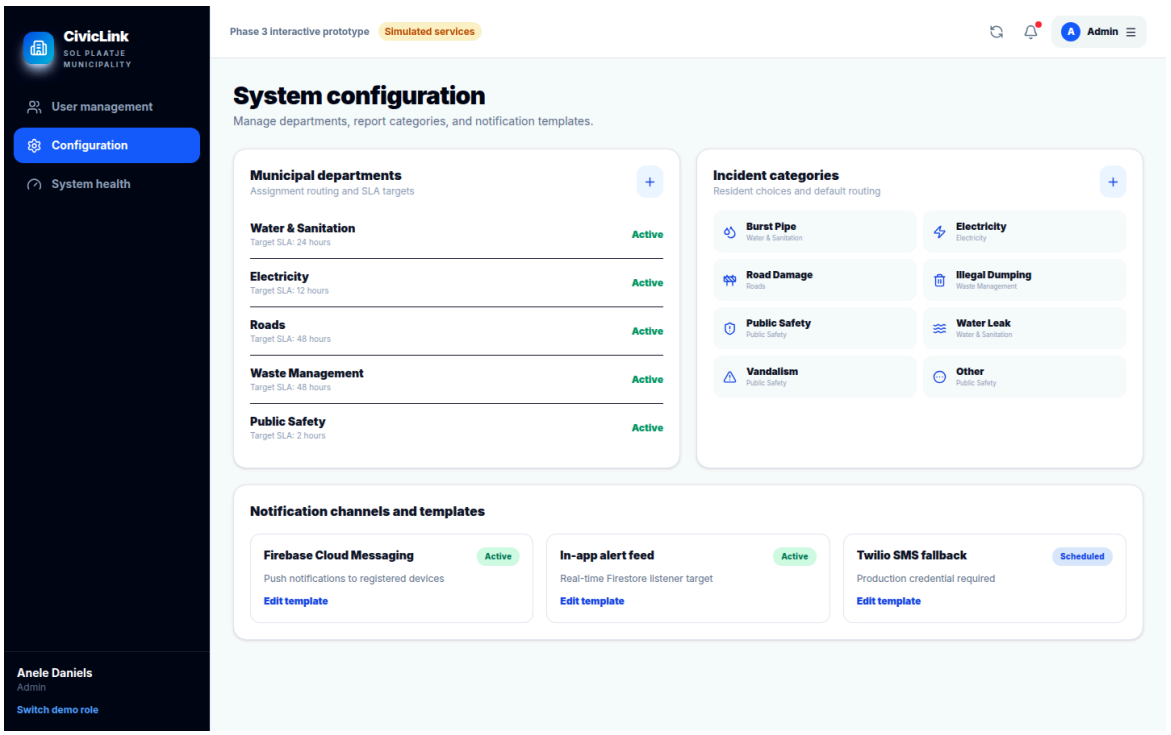


Figure 18: Department and Category Configuration

The Configuration screen allows System Administrators to manage the operational parameters that govern CivicLink's routing and notification behaviour. The Departments panel lists all municipal departments (Water & Sanitation, Electricity, Roads & Infrastructure, Waste Management, Public Safety), each with an editable SLA target in hours, a default notification template, and an active/inactive toggle that controls whether the department appears as an assignment option for officials.

The Categories panel manages the eight incident categories available to residents during report submission. Each category is mapped to a default routing department, ensuring that a 'Burst Pipe' report is automatically suggested for assignment to Water & Sanitation rather than requiring the official to select manually. Category labels and icons can be updated to reflect changes in municipal naming conventions. Both panels support the addition of new entries, allowing CivicLink to be extended to cover additional service types as the municipality's operational scope evolves.

3.19 System Health and Logs

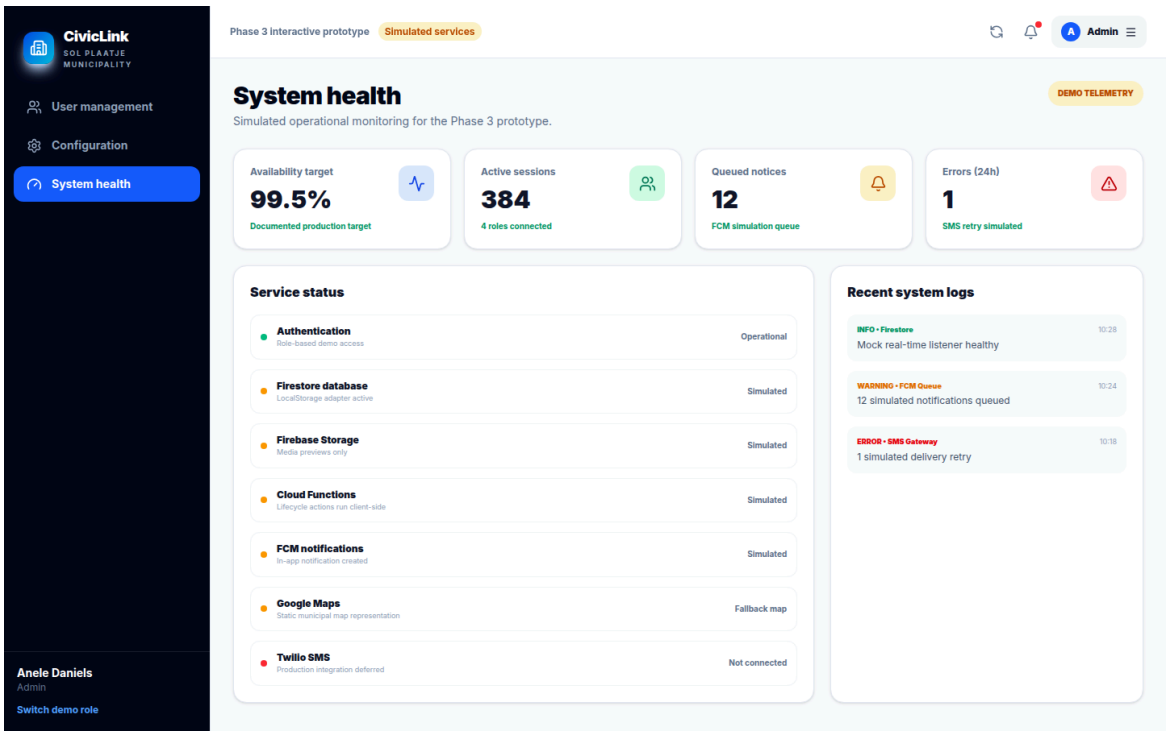


Figure 19: System Health and Logs

The System Health screen provides System Administrators with a live view of the platform's operational status and integration health. The screen clearly labels all telemetry as simulated prototype data while accurately representing the documented production architecture targets: 99.5% system availability, five-second report visibility SLA, FCM push notification queue status, active user sessions, and error event count.

Integration status indicators show the connection state of each external service — Firebase Auth, Firestore, Firebase Storage, Cloud Functions, FCM, Google Maps API, and Twilio SMS — confirming whether each service is operational, degraded, or unreachable. A recent operational logs panel displays timestamped system events including successful report submissions, FCM dispatch confirmations, authentication events, Cloud Function execution results, and any errors or warnings. In production, this data feeds from Firebase's native monitoring and the SystemLogs Firestore collection documented in Section 4. The health dashboard enables proactive detection of integration failures before they impact resident reporting or municipal operations.

4. Database Design

CivicLink targets Google Cloud Firestore as its production database — a NoSQL, document-oriented, real-time database organised into Collections of Documents. Firestore document IDs serve as primary keys, and cross-collection references stored as string fields serve as foreign key equivalents. Data integrity is enforced through Firebase Security Rules and Cloud Function validation rather than database-level constraints.

The Phase 3 prototype mirrors this schema precisely using a mock adapter backed by browser LocalStorage, allowing the client-side data model to be substituted with live Firebase services during production integration without changes to the application interface layer. The following sections document all ten entities, their attributes, data types, and constraints.

4.1 Entity: Users

The Users collection stores all registered CivicLink accounts across all four roles. A single collection is used rather than role-specific sub-collections, with the role field governing access control decisions at the application and security-rules level.

Entity: Users

Description: All registered CivicLink users — Residents, Municipality Officials, Field Workers, and System Admins.

Attribute	Data Type	Constraint	Description
userId	String	PK, NOT NULL	Auto-generated Firestore document ID. Displayed externally as a formatted user reference.
name	String	NOT NULL	Full display name of the user as entered during registration.
email	String	UNIQUE	Email address used for Firebase Authentication. Must be unique across all accounts.
phone	String	Optional	Mobile phone number for Twilio SMS fallback notifications. Not required for registration.
role	Enum	NOT NULL	Access role. One of: RESIDENT, OFFICIAL, FIELD_WORKER, ADMIN. Determines interface routing on login.
department	String	Optional	Municipal department for OFFICIAL and FIELD_WORKER roles. FK reference to Departments.departmentId.
location	GeoPoint	Optional	Resident's approximate home GeoPoint (lat, lng). Used to filter geo-targeted alert delivery.

suburb	String	Optional	Human-readable suburb name derived from location. Used alongside GeoPoint for alert targeting.
fcmToken	String	Optional	Firebase Cloud Messaging device token. Refreshed on each app launch. Required for push notification delivery.
isActive	Boolean	NOT NULL	Account status. Set to false by an Admin to deactivate access without deleting historical data.
createdAt	Timestamp	NOT NULL	Server-side timestamp set at account creation.
lastLogin	Timestamp	Optional	Server-side timestamp updated by Cloud Function on each successful authentication event.

4.2 Entity: Reports

The Reports collection is the central operational entity of CivicLink. Every resident-submitted incident is represented as a document in this collection, with all lifecycle state changes recorded in-place via the status field and fully audited in the StatusHistory sub-collection.

Entity: Reports

Description: All incident reports submitted by residents. The primary operational entity linking all other collections.

Attribute	Data Type	Constraint	Description
reportId	String	PK	Auto-generated Firestore document ID. Formatted as CL-YYYY-NNNN for resident-facing display.
userId	String	FK → Users	Reference to the userId of the resident who submitted the report.
category	Enum	NOT NULL	Incident category. One of: BURST_PIPE, ELECTRICITY, ROAD_DAMAGE, ILLEGAL_DUMPING, PUBLIC_SAFETY, WATER_LEAK, VANDALISM, OTHER.
description	String	NOT NULL	Resident-provided incident description. Maximum 300 characters. Validated server-side by Cloud Function.
location	GeoPoint	NOT NULL	GPS coordinates (latitude, longitude) automatically captured at submission time.
address	String	NOT NULL	Human-readable address derived from the GeoPoint via Google Maps Geocoding API.

mediaUrls	Array<String>	**Optional**	Firebase Storage download URLs for attached photos or video. Zero to three entries permitted.
severity	Enum	**NOT NULL**	Resident-assigned severity. One of: HIGH, MEDIUM, LOW. Inherited as task priority on assignment.
status	Enum	**NOT NULL**	Current lifecycle status. One of: SUBMITTED, UNDER_REVIEW, IN_PROGRESS, ESCALATED, RESOLVED, CLOSED.
department	String	**Optional**	Responsible municipal department. Set by an Official at assignment. FK reference to Departments.departmentId.
assignedTo	String	**FK → Users**	userId of the Field Worker currently assigned to handle this report. Null until assigned.
isDuplicate	Boolean	**NOT NULL**	Set to true by the deduplication Cloud Function if a matching open report exists within 100 metres.
parentReportId	String	**FK → Reports**	reportId of the original incident if this is a duplicate. Null for non-duplicate reports.
createdAt	Timestamp	**NOT NULL**	Server-side timestamp set at submission.
updatedAt	Timestamp	**NOT NULL**	Server-side timestamp updated by Cloud Function on every status change or data modification.
resolvedAt	Timestamp	**Optional**	Timestamp set when status transitions to RESOLVED. Used to calculate resolution time for analytics.

4.3 Entity: Alerts

The Alerts collection stores all municipality-broadcast notifications. Documents in this collection drive both the official Alerts Feed visible to residents and the delivery records stored in the AlertRecipients collection.

Entity: Alerts

Description: Municipality alerts broadcast to residents. Supports geo-targeted and system-wide emergency announcements.

Attribute	Data Type	Constraint	Description
alertId	String	**PK**	Auto-generated Firestore document ID.
createdBy	String	**FK → Users**	userId of the Municipality Official who composed and broadcast this alert.
type	Enum	**NOT NULL**	Alert category. One of: EMERGENCY, ELECTRICITY, WATER, ROADS, GENERAL, OTHER.
title	String	**NOT NULL**	Short alert headline. Maximum 100 characters.
message	String	**NOT NULL**	Full alert body with resident guidance or instructions. Maximum 500 characters.
targetSuburbs	Array<String>	**NOT NULL**	List of target suburb names (e.g. ['Galeshewe', 'Homestead']). Used to filter FCM token delivery.
targetGeo	GeoPolygon	**Optional**	GeoJSON polygon defining the precise geographic boundary when suburb-level targeting is insufficient.
channels	Array<String>	**NOT NULL**	Delivery channels selected for this broadcast. One or more of: FCM, IN_APP, SMS.
recipientCount	Number	**Optional**	Total unique users reached. Set by Cloud Function after broadcast completes.
isActive	Boolean	**NOT NULL**	Controls visibility in the resident Alerts Feed. Set to false to archive without deleting.
createdAt	Timestamp	**NOT NULL**	Server-side timestamp set at broadcast.
expiresAt	Timestamp	**Optional**	Scheduled expiry time. A Cloud Function automatically archives the alert and sets isActive to false.

4.4 Entity: Tasks

The Tasks collection stores all field worker assignments. One task document is created per report assignment event. If a report is reassigned, the original task is updated to REASSIGNED status and a new task document is created for the replacement worker.

Entity: Tasks

Description: Field worker task assignments. One task is created per report assignment and tracks completion from dispatch to resolution.

Attribute	Data Type	Constraint	Description
taskId	String	**PK**	Auto-generated Firestore document ID.
reportId	String	**FK → Reports, NOT NULL**	Reference to the source report that generated this task.
assignedTo	String	**FK → Users, NOT NULL**	userId of the Field Worker assigned to this task.
assignedBy	String	**FK → Users, NOT NULL**	userId of the Municipality Official who created the assignment.
department	String	**NOT NULL**	Responsible department. Denormalised from the Report for efficient field worker queries.
priority	Enum	**NOT NULL**	Task priority inherited from the report severity. One of: HIGH, MEDIUM, LOW.
status	Enum	**NOT NULL**	Task status. One of: ASSIGNED, ACCEPTED, IN_PROGRESS, COMPLETED, REASSIGNED.
notes	String	**Optional**	Operational guidance from the assigning official and completion notes from the field worker.
completionPhotos	Array<String>	**Optional**	Firebase Storage URLs of photographs submitted by the field worker as proof of resolution.
assignedAt	Timestamp	**NOT NULL**	Server-side timestamp of task creation.
startedAt	Timestamp	**Optional**	Timestamp set when the field worker accepts and begins the task.
completedAt	Timestamp	**Optional**	Timestamp set on task completion. Used to calculate field worker performance metrics.

4.5 Entity: Comments

The Comments collection stores the internal communication thread attached to each report. Comments are visible only to Municipality Officials and Field Workers and are not exposed to residents. They support structured communication between departments and field teams during incident resolution.

Entity: Comments

Description: Internal notes and communication attached to incident reports. Visible to Officials and Field Workers only.

Attribute	Data Type	Constraint	Description
commentId	String	**PK**	Auto-generated Firestore document ID.
reportId	String	**FK → Reports, NOT NULL**	Reference to the report this comment is attached to.
authorId	String	**FK → Users, NOT NULL**	userId of the Official or Field Worker who posted the comment.
authorName	String	**NOT NULL**	Display name of the author at the time of posting. Denormalised for read performance.
authorRole	Enum	**NOT NULL**	Role of the author. One of: OFFICIAL, FIELD_WORKER. Controls comment display styling.
text	String	**NOT NULL**	Comment body text. Maximum 1 000 characters.
createdAt	Timestamp	**NOT NULL**	Server-side timestamp set at posting time.

4.6 Entity: AlertRecipients

The AlertRecipients collection acts as a junction and delivery-log table between Alerts and Users, recording every individual notification delivery event. This enables the municipality to track reach, delivery confirmation, and read-receipt rates per broadcast.

Entity: AlertRecipients

Description: Junction collection logging individual alert delivery events. Enables per-broadcast reach and read-receipt analytics.

Attribute	Data Type	Constraint	Description
recordId	String	**PK**	Auto-generated Firestore document ID.

alertId	String	FK → Alerts, NOT NULL	Reference to the broadcast alert that generated this delivery record.
userId	String	FK → Users, NOT NULL	userId of the resident who received the alert.
deliveredAt	Timestamp	NOT NULL	Timestamp of successful FCM or SMS delivery to the recipient's device.
readAt	Timestamp	Optional	Timestamp of when the resident opened the alert in the app. Null until the alert is viewed.
channel	Enum	NOT NULL	Delivery channel used for this record. One of: FCM, IN_APP, SMS.

4.7 Entity: Departments

The Departments collection is a configuration entity managed by System Administrators. It defines the municipal departments available for report assignment and stores their SLA resolution targets.

Entity: Departments

Description: Municipal department definitions including SLA targets. Managed by System Admins via the Configuration screen.

Attribute	Data Type	Constraint	Description
departmentId	String	PK	Auto-generated Firestore document ID.
name	String	UNIQUE, NOT NULL	Department display name (e.g. 'Water & Sanitation', 'Electricity', 'Roads & Infrastructure').
slaHours	Number	NOT NULL	Target resolution time in hours for incidents assigned to this department. Used for SLA compliance analytics.
isActive	Boolean	NOT NULL	Controls whether this department appears as an assignment option. Inactive departments are hidden from officials.

4.8 Entity: Categories

The Categories collection defines the incident types available to residents during report submission. Each category is mapped to a default routing department, supporting intelligent pre-assignment suggestions for officials.

Entity: Categories

Description: Resident-facing incident categories with default department routing. Managed by System Admins via the Configuration screen.

Attribute	Data Type	Constraint	Description
categoryId	String	**PK**	Auto-generated Firestore document ID.
name	String	**UNIQUE, NOT NULL**	Category display name shown to residents (e.g. 'Burst Pipe', 'Illegal Dumping').
departmentId	String	**FK → Departments**	Default routing department for this category. Used to pre-populate the assignment suggestion.
isActive	Boolean	**NOT NULL**	Controls whether this category appears in the resident submission form. Inactive categories are hidden.

4.9 Entity: StatusHistory

The StatusHistory collection provides an append-only audit trail of every status change applied to a report throughout its lifecycle. It supports the resident-facing audit display in the Report Detail screen and provides the data basis for investigation of delayed or disputed resolutions.

Entity: StatusHistory

Description: Append-only audit log of all report status transitions. Supports resident transparency and internal accountability.

Attribute	Data Type	Constraint	Description
historyId	String	**PK**	Auto-generated Firestore document ID.
reportId	String	**FK → Reports, NOT NULL**	Reference to the report whose status changed.
status	Enum	**NOT NULL**	The resulting status after this transition (SUBMITTED, UNDER_REVIEW, IN_PROGRESS, ESCALATED, RESOLVED, CLOSED).
changedBy	String	**FK → Users**	userId of the actor who triggered the status change. May be a Cloud Function system user for automated transitions.
note	String	**NOT NULL**	Reason or description of the status change, required for accountability. Shown in the resident's audit trail.
changedAt	Timestamp	**NOT NULL**	Server-side timestamp of the status change event. Never editable after creation.

4.10 Entity: SystemLogs

The SystemLogs collection records operational events from Cloud Functions and service integrations. It populates the System Health and Logs screen accessible to System Administrators and supports incident diagnosis and audit compliance.

Entity: SystemLogs

Description: Operational event log from Cloud Functions and service integrations. Supports system monitoring and audit compliance.

Attribute	Data Type	Constraint	Description
logId	String	**PK**	Auto-generated Firestore document ID.
level	Enum	**NOT NULL**	Severity level of the event. One of: INFO, WARNING, ERROR.

service	String	**NOT NULL**	The integration or Cloud Function that produced this log entry (e.g. 'FCM', 'Firestore', 'Twilio').
message	String	**NOT NULL**	Human-readable description of the operational event.
createdAt	Timestamp	**NOT NULL**	Server-side timestamp of the event. Log entries are never modified after creation.

5. Entity Relationship Summary

The following table summarises all entity relationships in the CivicLink database, including cardinality and the field that implements each foreign key reference. Together, these relationships preserve end-to-end traceability from resident report submission through official assignment, field worker task completion, status audit, alert broadcast, and system administration.

From Entity	Relationship	To Entity	Cardinality	Foreign Key Reference
Users	submits	Reports	1 : 0..*	Reports.userId → Users.userId
Users	creates	Alerts	1 : 0..*	Alerts.createdBy → Users.userId
Users	receives	AlertRecipients	1 : 0..*	AlertRecipients.userId → Users.userId
Users	is assigned	Tasks	1 : 0..*	Tasks.assignedTo → Users.userId
Users	writes	Comments	1 : 0..*	Comments.authorId → Users.userId
Reports	generates	Tasks	1 : 0..1	Tasks.reportId → Reports.reportId
Reports	has	Comments	1 : 0..*	Comments.reportId → Reports.reportId
Reports	audited by	StatusHistory	1 : 0..*	StatusHistory.reportId → Reports.reportId
Reports	duplicate of	Reports	0..1 : 0..*	Reports.parentReportId → Reports.reportId
Alerts	delivered via	AlertRecipients	1 : 0..*	AlertRecipients.alertId → Alerts.alertId
Departments	routes	Categories	1 : 0..*	Categories.departmentId → Departments.departmentId
Departments	handles	Reports / Tasks	1 : 0..*	Reports.department / Tasks.department (string ref)

These relationships ensure that every incident report can be traced from initial submission through departmental assignment and field worker task completion, with a full status audit trail and associated internal commentary. The alert delivery model similarly provides end-to-end traceability from broadcast composition through FCM or SMS delivery to individual resident read receipts. The Departments and Categories configuration entities underpin the intelligent routing that eliminates the manual assignment overhead identified as a key inefficiency in the Phase 1 stakeholder interview.

6. Conclusion

Phase 3 of the CivicLink project delivers two interconnected outcomes that together form a decision-complete foundation for production development and deployment at Sol Plaatje Municipality in Kimberley.

The graphical user interface prototype, documented across nineteen figures in Section 3, demonstrates the complete operational lifecycle of the CivicLink platform across all four user roles. Every functional requirement identified in the Phase 1 stakeholder interview — GPS-enabled incident reporting, real-time status tracking, geo-targeted alert broadcasting, digital field worker task management, and operational analytics — is represented by a working, interactive interface. The shared prototype state ensures that actions performed in one role's interface are immediately visible in others, faithfully representing the real-time synchronisation that Firebase Firestore and Cloud Functions will deliver in production.

The database design, documented across ten entities in Section 4, provides the complete schema required to migrate from the prototype's LocalStorage adapter to live Firebase services. Primary keys, foreign key references, data types, constraints, and cardinalities are fully specified, with production architecture considerations — including Firestore Security Rules, Cloud Function validation, and FCM token management — incorporated into the field-level descriptions. The StatusHistory and SystemLogs entities extend the schema beyond the Phase 2 class diagram to meet the audit and operational monitoring requirements identified during detailed design.

Taken together with the systems analysis of Phase 1 and the architectural and UML modelling of Phase 2, the Phase 3 deliverables provide a comprehensive, academically documented, and technically actionable blueprint for the delivery of a municipal reporting and community intelligence platform capable of transforming the way Sol Plaatje Municipality communicates with and responds to the residents of Kimberley.

— End of Phase 3 Report —